ÜBERBLICK

Projektüberblick

Was das Projekt ist, wie es aufgebaut ist und wie die Dokumente zusammenhängen

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	`C8205-Interpreter-Projektueberblick.md`	0.95-008
Stand	11.11.2026	
Umfang	11 Seiten, A4 hoch	
Erzeugt mit	`erzeuge_projektueberblick_pdf.py`	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

Inhaltsverzeichnis

1. Ziel und Einordnung . . . 3
2. Zwei-Wege-Architektur: Python-Interpreter und Claude-Interpreter . . . 3
3. Teilprojekt Assembler . . . 4
3a. Teilprojekt BIS . . . 5
3b. Experimenteller Prototyp C8205Z (Zusatztrommeln) . . . 5
4. Virtuelle Hardware im Überblick . . . 5
4a. Zwei Betriebsarten: TEST und GRAFIK . . . 6
5. Testkonzept: Trommeldatei, Tastenfolge, Testmarken, automatische Prüfung . . . 6
6. Dokumentenlandkarte (.md-Dateien und Code) . . . 7
7. PDF-Dokumente im Überblick . . . 10
8. Stand und offene Punkte . . . 11

C 8205 – Projektüberblick

Version: 0.95-008

Dieses Dokument richtet sich an Menschen, die das Projekt nicht im Detail verfolgt haben, und beschreibt zusammenhängend, was hier entsteht und wie die Einzeldokumente zueinander stehen. Es ersetzt keines der Fachdokumente, sondern führt in sie ein und verweist auf sie.

1. Ziel und Einordnung

Ziel des Projekts ist die **Simulation der Befehlsverarbeitung** des Cellatron C 8205, eines 1968 in der DDR gebauten Kleinrechenautomaten. Der C 8205 ist ein Einadreßautomat mit vergleichsweise geringem technischen Aufwand (rund 400 Transistoren, 2000 Dioden in der Zentraleinheit) und für einen Kleinrechner hoher Operationsgeschwindigkeit (bis zu rund 2000 verdrahtete Operationen je Sekunde bei zeitoptimaler Programmierung). Anstelle eines RAM-Speichers verwendet er eine Magnettrommel: Zugriffe auf einzelne Speicherzellen sind dadurch nicht beliebig schnell möglich, sondern hängen davon ab, wann die gewünschte Stelle der Trommel unter dem Lese-/Schreibkopf ansteht. Bedient wird der Rechner nicht über einen Bildschirm, sondern über ein Bedienpult mit Tasten, Schaltern und Lampen; Programme und Daten werden über Lochstreifen ein- und ausgegeben.

Die Simulation rekonstruiert diesen Rechner in zwei Schritten: Zunächst wird die Elektronik des Befehlsablaufs (die "vier Phasen" der Befehlsabarbeitung, phasenweise als Knotenautomat mit rund 80 Knoten beschrieben) aus Originalunterlagen so genau wie möglich als Regelwerk rekonstruiert. Darauf aufbauend wird dieses Regelwerk **interpretativ** ausgeführt — wortlautgetreu, ohne stillschweigende Vereinfachung: Ist eine Stelle des Regelwerks unklar, wird das als offener Punkt gemeldet statt eigenmächtig "repariert" ("laut scheitern"-Prinzip). Im Stapelbetrieb **TEST** werden alle Zeitabhängigkeiten der realen Hardware (Wortzeit, Wartezeit der Trommel) auf 0 gesetzt: Die zeitliche Reihenfolge der Ereignisse bleibt erhalten, es entstehen aber keine echten Wartezeiten. Der Modus **GRAFIK** (Abschnitt 4a) bildet die Trommelzeit dagegen nach (Wortzeit 104 µs, 32 Sektoren; Zeitmodell wählbar mit `--trommelzeit alt|echt|schnell`).

Das Gesamtprojekt gliedert sich dafür in drei Teilprojekte mit unterschiedlichem Zweck: Der **Interpreter** (Python, Wurzel des Projektverzeichnisses; Abschnitt 2) führt Maschinencode gegen das rekonstruierte Regelwerk aus, der **Assembler** (C, Unterverzeichnis `Assembler/`; Abschnitt 3) übersetzt lesbaren Assembler-Quelltext in genau diesen Maschinencode, und das **Bausteininterpretiersystem BIS** (Abschnitt 3a) ist das Programmsystem, das auf dem C 8205 selbst läuft. Interpreter und Assembler sind eigenständige Werkzeuge, die über den von ihnen erzeugten bzw. verarbeiteten Objektcode zusammenhängen, aber unabhängig voneinander entwickelt werden. Das BIS ist kein Werkzeug, sondern Anwendungs-Software für den C 8205; als Teilprojekt steht es gleichberechtigt neben den beiden anderen. Daneben gibt es als experimentellen Zusatzpfad den Prototyp der Zusatztrommeln C8205Z (Abschnitt 3b); er ist kein viertes Teilprojekt und verändert den Simulator nur mit dem Schalter `--C8205Z`.

2. Zwei-Wege-Architektur: Python-Interpreter und Claude-Interpreter

Ein und dasselbe Testformat (Abschnitt 5) kann auf zwei unabhängigen Wegen ausgeführt werden, die absichtlich **auf gleiche Art und Weise** arbeiten — derselbe Knotengraph, dieselbe Testdatei, dasselbe Protokollformat:

- **Python-Interpreter** (`c8205_interp.py`): führt den Knotengraphen deterministisch und automatisiert aus. Schnell und reproduzierbar, aber ohne Rückfragemöglichkeit bei unklaren Regelwerk-Stellen. Er liest den Knotengraphen und die Variablentabelle T2 zur Laufzeit direkt aus den Regelwerk-Dateien (nicht fest einprogrammiert); nur die Wirkung jeder einzelnen ANWEISUNG ist von Hand als Python-Funktion hinterlegt. Fehlt zu einer ANWEISUNG eine Funktion, meldet der Interpreter das als Stub statt zu raten.
- **Claude-Interpreter**: Eine KI erhält das komplette Regelwerk als Kontext sowie dieselbe Testdatei und arbeitet den Knotengraphen manuell, Knoten für Knoten, ab. Langsamer und nicht deterministisch reproduzierbar (abhängig von Modell und Sitzung), erlaubt dafür aber Rückfragen und Erläuterungen während der Abarbeitung.

Rollenverteilung: Der Python-Interpreter übernimmt die laufenden Testläufe. Der Claude-Interpreter dient als unabhängige Gegenprobe — weichen die Ergebnisse beider Wege bei einem Test voneinander ab, liegt ein Fehler in einem der beiden vor (oder eine tatsächlich unklare Stelle im Regelwerk) — und kommt gezielt bei Rückfragen sowie beim Durchspielen neuer oder geänderter Befehle zum Einsatz, bevor diese in `c8205_interp.py` übernommen werden. Diese Aufgabenteilung gilt modellneutral für jede ausführende KI, nicht nur für Claude.

Wie ein Test auf beiden Wegen gestartet wird, welche Dateien dem Claude-Interpreter hochzuladen sind und wie die Startanweisung aussehen kann, steht in `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 1b.

3. Teilprojekt Assembler

Der Assembler `c82ass` (Cross-Assembler für C8205/C8205Z, in C geschrieben, läuft unter Linux und Windows 10; die Versionsnummer nennt `c82ass --version`, die Stellen der Hochzählung stehen in `C8205-Versionierung.md`) ist ein eigenständiges Werkzeug, kein Interpreter oder Simulator: Er übersetzt lesbaren Assembler-Quelltext (`.asm`) in ladefähigen Objektcode (K1K2-Textformat) für das "Kleine Eingabeprogramm" des C 8205, denselben Maschinencode, den der Interpreter (Abschnitt 2) als Trommeldatei ausführt. Aufruf, Sprachdefinition, Anweisungen, Makros und Beispiel stehen in der Assemblerdokumentation (`Assembler/Dokumentation/`, die `.md` ist maßgeblich); hier nur das Wesentliche:

- **Vier Pässe:** Pass 0 löst die Makros auf (`c82ass.makros`), Pass 1 ermittelt die Literaltabelle, Pass 2 die Symboltabelle (beide im `.log`, die Symbole zusätzlich in der `.sym`), Pass 3 erzeugt die Kennzeichenbefehle im Objektfile (`.obj`) sowie `.mqo`, `.dmp` und `.test`. Zwischen Pass 2 und Pass 3 läuft der **Optimierer** (standardmäßig an, `--no-opt` schaltet ihn ab): Er verlegt Literale in freie Zellen, verschiebt Blöcke (`begin RELOC-EQU`, `begin RELOC-ALL`) und ordnet Makro-Rümpfe um (Assemblerdokumentation, Abschnitt 5). Mit `--pass0` läuft nur Pass 0.
- **Sprache:** Maschinenbefehle (3 Oktalziffern, oder Namen mit vorangestelltem Punkt wie `.LD` über die Tabelle `c82ass.opsym`) gemischt mit Assembleranweisungen: Datendefinition (`db`/`dbn` Bereich frei/auf Null, `dw` Wert, `dwb` Bitfolge, `dwh` hexadezimal, `dwf` Festkommabruch, `dwg` Gleitkommazahl, `da` Adresse, `dt`/`dto`/`dtm` Text als R300-Zeichenfolge, `kat4`/`kat5`/`kat6` komprimierter Text, `dz` Sonderzeichen), Adressdefinition (`equ` relativ, `equ10`/`equ40`/`equ100` Ausrichtung, `abs` absolut, `spa` Sektoradresse), Programmrahmen (`start`/`end`) und zusammenhängende Folgen (`begin`/`bend`). Adresssymbole (bis 32 Zeichen, mit Doppelpunkt abgeschlossen) und Literale (`#Werteausdruck`, `@Adressausdruck`, `!Name` als benannte Arbeitszelle; vom Assembler automatisch auf einem eigenen Speicherplatz abgelegt) ergänzen die Sprache.
- **Makro-Präprozessor:** `MACRO`/`MEND`-Blöcke mit Parameterersetzung (`$1`, `$2`, …), bedingter Codegenerierung (`$IF`/`$ELIF`/`$ENDIF` mit bis zu vier über `|`/`&` verknüpften Vergleichsblöcken) und generischen, mehrfach verwendbaren Labels über einen `%`-Platzhalter, der bei jedem Aufruf durch eine fortlaufende Nummer ersetzt wird. Dazu kommen weitere `$`-Direktiven, Funktionen, benannte Parameter und Schlüsselwortparameter (Assemblerdokumentation, Abschnitt 4). Die mitgelieferten Makros beschreiben `Assembler/Dokumentation/C8205_Standardmakros_c82ass.md` (Standardmakros) und `C8205_ZT_Makros_Uebersicht.md` (ZT-Makros).
- **Programmprojekte und Generatoren:** Unter `Assembler/` liegen neben dem Quelltext des Assemblers die Assemblerprogramme selbst, je Projekt ein Ordner (`Projekt_1/`, `Projekt_2/` mit dem BIS-Minimalprogramm, `Projekt_Festkomma/`, `Projekt_Gleitkomma/`, `Proj_FeldSortieren/`, `Projekt_ZT_LOADER/` für die Zusatztrommeln, `Claude/` mit kleinen Prüfprogrammen je Assemblermerkmal). Größere Programme (ab etwa 30 Zellen mit Schleife, Feld, Unterprogramm oder Ein-/Ausgabe) entstehen als Python-Generator im Unterordner `gen/` des Programms; die gemeinsame Bibliothek `Assembler/Claude/gen_lib/c82gen.py` (Messung und Einsatzregel: `gen_lib_messung.md`) macht Python zur Makrosprache. Die erzeugte `.asm` bleibt maßgeblich und lesbar (Programmierhandbuch 5.7). Der Assembler selbst wird mit `Assembler/Regressionstest/` geprüft (Beschreibung: `LIESMICH.md` darin; der Ordner liegt als `Regressionstest.zip` im Projektstamm).
- **Erzeugte Dateien:** Wichtig sind `.obj` (Objektcode) und `.test` (Trommeldatei für den Interpreter, mit Platzhaltern für Testmarken und Tastenfolge); `.log`, `.mqo`, `.sym` und `.dmp` sind Tabellen, Listing und Hilfsdateien. Die vollständige Liste steht in `Projektstruktur.md`, Abschnitt 3.
- **Brücke zum Interpreter:** Die zusätzlich erzeugte `.test`-Datei enthält die vollständige Trommeldatei im Testformat des Interpreters (`Adresse: Wert`, oktal, mit der jeweiligen Quelltextzeile als Kommentar) sowie Platzhalter-Definitionen für Testmarken und Tastenfolge und ggf. den Leerbefehl `164`. Diese Platzhalter müssen für den jeweiligen Fall ergänzt bzw. erweitert werden (Testmarken, Tastenfolge und Prüfungen: `Durchführung_eines_Tests.md`, Abschnitte 2 bis 3b). Den Standardweg erledigt `Assembler/asm_und_test.sh`: Es assembliert, ergänzt die `.test` (Sollwerte aus dem `;#PRUEFE`-Block des `.asm`) und startet den Interpreter. Ältere Testaufträge liegen als `Tests/asm_NNNN_<Programmname>.txt` (Namensschema: `Projektstruktur.md`, Abschnitt 2). Der Code selbst stammt unverändert aus dem Assemblerlauf (`Durchführung_eines_Tests.md`, Abschnitt 2a).

3a. Teilprojekt BIS

Das Bausteininterpretiersystem (BIS; Herausgeber VEB Kombinat Zentronik) ist ein **offenes Unterprogrammsystem** für den C 8205 und das dritte Teilprojekt. Anders als Interpreter und Assembler ist es kein Werkzeug auf dem Entwicklungsrechner, sondern Software, die auf dem C 8205 selbst abläuft. Der Rechner kennt in Hardware nur die Eingabe **eines Zeichens**; alles Weitere (Worte, Adressen und Dezimalzahlen einlesen, Multiplikation und Division, Gleitkomma, Ausgabe) leisten Bausteine des BIS. Es enthält:

- **Eingabebausteine, Rechenbausteine, Ausgabebausteine und Serviceprogramme** (vier Gruppen), alle zeitoptimal aufgebaut, je Baustein 67_8 bis 377_8 Speicherplätze, unter festen Adressen erreichbar,
- das **Kleine Eingabeprogramm (KEP)** auf Spur 0 (Adressen 0000_8 bis 0037_8), das nur die Zeichen 0, 1, Punkt, Doppelpunkt und Zwischenraum verarbeitet und mit dem das Minimalprogramm geladen wird,
- das **Minimalprogramm**, den Grundbaustein, der Befehle, Adressen und Konstanten über die Eingabekennzeichen K_1 und K_2 einliest.

Stand der Erfassung: Dokumentiert sind bisher die allgemeine Beschreibung, das KEP und das Minimalprogramm; weitere Bausteine sind noch nicht erfasst (`C8205-Interpreter-BIS.md`).

Beziehung zu den anderen Teilprojekten:

- Die Gleitkomma-, Festkomma- und Tetradendarstellung der Zahlen sowie ihre externe Darstellung sind Konventionen von Programmen, nicht der Rechner-Hardware. Die Übersicht und die Wortdeutungen (ganze Festkommazahl, interne Gleitkommadarstellung) stehen in `C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md`, die externe Darstellung und die Tetraden des BIS in `C8205-Interpreter-BIS.md` (Abschnitt BIS-1.1).
- Der Objektcode des Assemblers (K1K2-Format, Abschnitt 3) ist für das KEP bestimmt, also für den ersten Baustein des BIS.
- Interpreter und Assembler kennen das BIS nicht: Es ist kein Teil der simulierten Hardware. Bausteine lassen sich aber als Trommeldatei eines Tests im Interpreter ausführen, und das geschieht auch: Das Minimalprogramm liegt als Assemblerprojekt in `Assembler/Projekt_2/` (Fassung v6, daneben v7 mit optimierter Ziffernschleife), seine Testaufträge in `Tests/` (`pj2-bis_v6_*`, `pj2-bis_v7_*`), fertige Lochstreifen in `Lochstreifen/`. Der Ordner `Bausteininterpretiersystem BIS/` selbst enthält nur die Dokumentation.

Einzelheiten stehen nicht hier, sondern in `C8205-Interpreter-BIS.md`. Das Format der Objektdatei, die das KEP liest, und der Weg vom Vorgabebild zum Testauftrag stehen in `C8205-Interpreter-Objektdatei_Listing.md` (ebenfalls im Ordner `Bausteininterpretiersystem BIS/markdown-dateien/`).

3b. Experimenteller Prototyp C8205Z (Zusatztrommeln)

Der Ordner `C8205Z/` enthält einen **experimentellen Prototyp** der vier Zusatztrommeln des C 8205Z. Er ist kein Teilprojekt: Ohne den Aufrufschalter `--C8205Z` bleibt der Simulator unverändert, und der Assembler wird dafür nicht verändert. Eckdaten, Zeitmodell, Befehle (`5k00 030`, `zzz0 bbbb 601`, `zzz0 bbbb 661`) und Modulspezifikation stehen in `C8205Z/zt/Zusatztrommel_Zeitmodell.md`; das ZT-Repository und der Urlader ZIL in `C8205Z/zt/ZT-Repository_Aufgabenstellung.md`. Die zugehörigen Assemblerprogramme liegen im Projekt ZT-Loader (`Assembler/Projekt_ZT_LOADER/`). Der Prototyp hat ein C-Modul (`C8205Z/zt_modul/`) mit Python-Erweiterungsmodul und eigene Tests (`C8205Z/Tests_Z/`, Aufruf `bash C8205Z/Tests_Z/zt_tests.sh`); im Simulator selbst hängt er an `c8205_zt.py` (TEST) und `grafik-interpreter-module/grafik_zt.py` (GRAFIK). Verzeichnisse und Tests: `C8205-Projektstruktur.md`, Abschnitt 3b.

4. Virtuelle Hardware im Überblick

Der reale C 8205 (ECHT) besteht aus Stromversorgung, Zentraleinheit (Rechenwerk und Leitwerk) und Peripherie: Magnettrommelspeicher, Bedienpult mit Tastatur/Lampenfeld/Bahnsperrenschaltern, zwei Lochbandlesern, einem Lochbandstanzer und einem Schreibwerk für alphanumerische Ein-/Ausgabe. Für die Simulation wird davon **virtualisiert**, was ein Testlauf am Bildschirm bzw. in Dateien braucht — mit denselben Kanal- und Adresszuordnungen wie beim Original, aber ohne mechanische Bauteile:

- **Bedienpult:** Tasten (E, G, M, Ü, ST u. a.) und Schalter wirken als Tastenfolge auf Register und Flags, statt tatsächlich gedrückt zu werden; Lampen werden durch die protokollierten Flag-Werte ersetzt.
- **Magnettrommel:** statt der physischen Trommel eine persistente Textdatei mit allen 4096 Zellen (`Trommel/trommel.txt`, Adresse:Wert je Zeile, oktal). Sie behält ihren Inhalt über Tests hinweg. Ein Test gibt die

Zellen, die er braucht, vorab als Textdatei (Adresse:Wert je Zeile, oktal) mit; sie werden auf die Trommel geschrieben und überlagern deren Inhalt.

- **Lochbandleser (2, Kanäle K1/K2):** je Leser eine virtuelle Streifendatei; ist keine angegeben und fordert das Programm dennoch eine Eingabe an, bleibt der Lauf mit gesetztem Flag E stehen wie beim Original ohne eingelegten Streifen — kein Fehler.
- **Lochbandstanzer (Kanal K1):** Ausgaben werden zeilenweise binär (Spuren w v u p 8 4 2 1) in eine Ausgabedatei geschrieben, statt real gestanzt zu werden.
- **Schreibwerk (Ausgabe über Kanal K2):** zeichenweise Textausgabe, mit WRZL (Wagenrücklauf Zeilenvorschub) als Zeilenendekennung, statt auf eine Schreibmaschine: in die Datei `sw-ausgabe.txt`, wenn dem Interpreter eine Datei zugeordnet ist (`--sw-ausgabe`), sonst auf das Terminal.
- **Tastatur und Schreibwerk-Eingabe (Kanal K0):** Das Zeichen kommt im Standardfall von der Tastatur des Host-Systems, mit `--k0 datei=PFAD` aus einer Datei.

Hilfsprogramme für die Lochstreifen: Drei eigenständige Python-Programme arbeiten mit denselben virtuellen Lochstreifen wie der Interpreter. `c8205_lochstreifen_anzeige.py` schreibt einen Lochstreifen (etwa die Ausgabe von `--stanzer1`) als lesbaren Text in einem Fenster aus; `c8205_lochstreifen_eingabe.py` erzeugt aus Tastatureingaben einen Lochstreifen, den man dem Interpreter mit `--leser1`/`--leser2` übergeben kann; `c8205_lochstreifen_umwandlung.py` wandelt ohne Fenster eine Textdatei in einen virtuellen Lochstreifen um und umgekehrt. Alle drei gehören zum Interpreter-Teilprojekt, werden vom Interpreter aber nicht aufgerufen (Doku: `C8205-Interpreter-Lochstreifen-Hilfsprogramme.md`).

Jede virtuelle Datei wird nur angelegt, wenn der jeweilige Test das Gerät tatsächlich benutzt, und am Testende zum Download bereitgestellt. Ausnahme ist die Trommeldatei: Sie muss bei jedem Lauf vorhanden sein und wird vom Interpreter nie angelegt (Hilfsskript `testscripte/erzeuge_leere_trommel.py`).

4a. Zwei Betriebsarten: TEST und GRAFIK

GRAFIK: Der Python-Interpreter läuft in zwei Betriebsarten. **TEST** ist der Stapelbetrieb für automatische Tests (Trommeldatei, Tastenfolge, Testmarken). **GRAFIK** (Vorgabe ohne Angabe) bedient den Rechner wie am Pult: Bedienpult- und Schreibwerk-Fenster, Töne, echte Trommel. Beschreibung: `C8205-Interpreter-GRAFIK_Beschreibung.md`; Prüfung: `C8205-Interpreter-GRAFIK_Pruefverfahren.md`. Treiber ist `c8205_grafik.py` (Module in `grafik-interpreter-module/`, die Trommel mit Hintergrund-Thread als C-Modul in `c-module/`; Prüfskripte `testscripte/grafik_*`, alle über `testscripte/grafik_alle_tests.sh`). Die virtuellen Geräte von GRAFIK liegen in `virtuelle_Geraete/`, die des TEST-Interpreters in `Test-virtuelleGeraete/`.

5. Testkonzept: Trommeldatei, Tastenfolge, Testmarken, automatische Prüfung

Jeder simulierte "Rechnerlauf" ist als **Test** definiert, unabhängig davon, ob er anschließend vom Python- oder vom Claude-Interpreter ausgeführt wird. Ein Test besteht aus:

- **Trommeldatei** (Abschnitt `Trommeldatei:` der Testdatei): der für den Test relevante Ausschnitt des Speichers, zeilenweise als `Adresse: Wert` (beide oktal), mit Kommentaren (`#`) und Leerzeilen. Er wird vor dem Test auf die persistente Trommel geschrieben; nicht genannte Zellen behalten ihren bisherigen Inhalt (`Durchführung_eines_Tests.md`, Abschnitt 2c).
- **Tastenfolge:** definiert den Ausgangszustand von Registern und Flags vor Beginn der Befehlsschleife — welche Tasten/Schalter in welcher Reihenfolge betätigt werden (z. B. `H G R` für einen regulären Start), einschließlich der jeweiligen Wirkung auf Register und Flags.
- **Testmarken** (Zeilen mit `#…` in der Testdatei) für Diagnoseausgaben, Anfangswerte, Starthilfen, Unterbrechungen, Protokolltiefe und den Trommelzustand; sie haben keine Entsprechung in der realen Elektronik. Vollständige Liste und Wirkung: `Durchführung_eines_Tests.md`, Abschnitte 2 bis 2g.
- **Prüfungen** (optional, Abschnitt `Prüfungen:` am Ende der Testdatei): Sollwerte für Register, Flags und Speicherzellen sowie Dateiprüfungen für die Stanzer- und die Schreibwerk-Ausgabe; das Gesamtergebnis lautet *Test wahr* oder *Test falsch* (`Durchführung_eines_Tests.md`, Abschnitte 3a und 3b).

Automatische Prüfbarkeit: Weil derselbe Test mit demselben Format deterministisch (Python) oder als Gegenprobe (Claude) ausgeführt werden kann, lassen sich für jeden Test **Sollwerte** festhalten — Endzustand von Register/Flags, erwartete Speicherinhalte, erwartete Ausgaben. Damit wird jeder Testlauf zu einem Regressionstest: Ein neuer Lauf wird automatisch gegen die zuletzt bestätigten Sollwerte verglichen, ohne dass ein Mensch das Protokoll erneut Zeile für Zeile lesen muss. Voraussetzung ist ein definierter Trommelstand: Weil die

Trommel Tests überdauert, gelten Sollwerte für einen Lauf auf leerer Trommel oder für einen Test, der alle benutzten Zellen selbst nennt (Abschnitt 2c der Testbeschreibung). Der Gesamtlauf `testscripte/compute_all_tests.sh` führt alle Testaufträge in `Tests/` aus (zuletzt 168 Läufe ohne Abweichung); Assemblerprogramme testet `Assembler/asm_und_test.sh` mit den Sollwerten aus dem `;#PRUEFE`-Block des `.asm`.

Ergänzend dazu prüft `c8205_befehlssweep.py` systematisch alle 512 möglichen Operationscodes (000–777 oktal) gegen den Knotengraphen: Für jeden Code wird ein Minimalprogramm gebaut und direkt über dieselbe Knotengraph-Engine wie `c8205_interp.py` ausgeführt. Das prüft nicht die numerische Richtigkeit jeder Formel — dafür bleiben die inhaltlichen Einzeltests zuständig —, sondern ob für jeden Code überhaupt eine ANWEISUNG-Implementierung vorhanden ist und der Automat sauber terminiert (kein Stub, kein Sackgassen-Knoten, keine erkennbare Endlosschleife): ein automatisierter Vollständigkeits-Regressionstest zusätzlich zu den inhaltlichen Einzeltests.

6. Dokumentenlandkarte (.md-Dateien und Code)

Datei	Rolle
`C8205-Einstieg_fuer_Neueinsteiger.md`	Orientierung für neue Leser des Gesamtprojekts (Interpreter, Assembler, BIS): was die Teile bedeuten, empfohlene Lesereihenfolge, was man zunächst nicht lesen muss, Zuordnung Frage → Dokument
`C8205-Interpreter-Überblick_und_Gesamtaufbau.md`	Wurzeldokument: Einordnung, Geräteaufbau, Rechen-/Leitwerk, die vier Phasen der Befehlsabarbeitung, Speicher-Timing und Leistungswerte
`C8205-Interpreter-Magnettrommel.md`	Speicherorganisation der Trommel (128 Spuren × 32 Sektoren), Interleaving/physische vs. logische Sektoradresse, Bahnsperrenschalter; Nachbildung im Interpreter als persistente Datei
`C8205-Interpreter-Bedienpult_mit_Schaltern.md`	Bedienelemente: Tasten, Schalter, Kanalanwahl, Wirkung auf Register/Flags
`C8205-Interpreter-Lochstreifenleser.md`	Hardwarebeschreibung Lochbandleser, -stanzer und Schreibwerk (Kanäle, Ablauf einer Zeichen-Ein-/Ausgabe)
`C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md`	Register (AC, BZ, AR, OR, BR), Flags, Maschinenwort- und Befehlswortaufbau (Bitfelder), Übersicht der Zahlendarstellungen
`C8205-Interpreter-Eingänge_der_Knoten.md`	Der Knotengraph selbst: alle Knoten (Konnektor/Phasenzuordnung/Anweisung/Test), Ein-/Ausgänge, Verzweigungen
`C8205-Interpreter-Beschreibung_der_Knoten.md`	Variablentabelle T2: Bedeutung jeder ANWEISUNG-nnn, die im Knotengraph referenziert wird
`C8205-Interpreter-Befehlsliste.md`	Generische Befehlsbeschreibung nach B/A/Z sowie die vollständige, nach Oktalcode geordnete Einzelbefehlsübersicht (alle 512 Operationscodes)
`C8205-Interpreter-Kodetabellen.md`	Zeichenkodetabelle: R300-Lochstreifenkode (Tabelle 1)
`Bausteininterpretiersystem BIS/markdown-dateien/C8205-Interpreter-BIS.md`	Teilprojekt BIS (Abschnitt 3a): Bausteininterpretiersystem, Kleines Eingabeprogramm (KEP) auf Spur 0, Minimalprogramm mit den Eingabekennzeichen K_1/K_2
`Bausteininterpretiersystem BIS/markdown-dateien/C8205-Interpreter-Objektdatei_Listing.md`	Objektdatei-Ausdrucke der Beispielprogramme (Vorgabebilder): Spalten einer Befehlszeile, Relokationskennung K1K2, vom Vorgabebild zum Testauftrag, Beispiel Maximum von drei Zahlen; liegt im BIS-Ordner

C 8205 – Projektüberblick

Quelle: C8205-Interpreter-Projektueberblick.md · im Wortlaut der Quelle

Datei	Rolle
C8205-Interpreter-Durchführung_eines_Tests.md	Das Testkonzept selbst (Abschnitt 5 dieses Dokuments): Trommeldatei, Tastenfolge, Testmarken, virtuelle Geräte, Protokollierung, Python-/Claude-Interpreter, Start eines Tests auf beiden Wegen (Abschnitt 1b)
C8205-Interpreter-Lochstreifen-Hilfsprogramme.md	Bedienung und Technik der drei Lochstreifen-Hilfsprogramme (Anzeige, Eingabe, Umwandlung): Format des virtuellen Lochstreifens, Zeichenlage, Irrung, Kopieren in die Zwischenablage, Installation unter Linux, Windows, macOS
C8205-Interpreter-GRAFIK_Beschreibung.md	Der Interpreter in der Betriebsart GRAFIK (Abschnitt 4a): Fenster (Bedienpult, Schreibwerk, HW-Check), Geräte, Töne, Takt, Aufruf und Grenzen
C8205-Interpreter-GRAFIK_Pruefverfahren.md	Prüfverfahren für GRAFIK: Läufe, Tests und Abnahme gegen das Soll der Beschreibung
C8205-Interpreter-Python_Interpreter.md	Bedienung von `c8205_interp.py`: Installation, Aufruf, Schalter, Fehlermeldungen, Beispiele; Vorgaben für die Bedienungsanleitung (PDF)
baz-wirkungstabelle.md	Cache bereits hergeleiteter Knotenpfade/Formeln je getestetem BAZ-Wert, um wiederholte Herleitung zu vermeiden
C8205-Projektstruktur.md	Wurzeldokument der Verzeichnisstruktur: welches Verzeichnis/welche Endung welchen Zweck hat, für Interpreter und Assembler
C8205-Programmierhandbuch_fuer_Claude.md	Nachschlagewerk zum Schreiben, Assemblieren und Testen von Assemblerprogrammen für den C 8205 (Wurzelordner); Regeln des Projektinhabers, Idiome, Festlegungen, Stolperfallen
C8205-Programmierhandbuch-Anhang_A_Trommelzeit_GRAFIK.md, C8205-Programmierhandbuch-Anhang_B_BIS_Anschluss.md	Anhänge zum Programmierhandbuch: Trommelzeit und GRAFIK-Zeitmessung (A), Anschluss an das BIS-Minimalprogramm (B)
C8205-Schleifenoptimierung_Eingabeschleife.md	Gerechnete Untersuchung der Lage der Ziffern-Eingabeschleife des BIS-Minimalprogramms auf der Trommel (nur Dokumentation)
CLAUDE.md	Karte für Claude (Wurzelordner): zuerst zu lesen; Schnellstart, Regeln des Projektinhabers, Zuordnung Aufgabe → Dokument
C8205-Versionierung.md	Wurzeldokument der Versionsnummern: Aufbau `y.xx-zzz`, welche Dateien eine tragen, wo sie steht und wann sie sich ändert
C8205-Interpreter-Projektueberblick.md	dieses Dokument: Ziel, Zwei-Wege-Architektur, Teilprojekte, virtuelle Hardware, Testkonzept, diese Dokumentenlandkarte, PDF-Übersicht, Stand
README.md, README.en.md	Einstiegsseiten für GitHub (deutsch, englisch) im Wurzelverzeichnis: fassen das Projekt auf einer Seite zusammen und verweisen auf die Dokumente; keine eigenen Regeln
C8205_Assembler_Dokumentation.md	Maßgebliche Assemblerdokumentation: Sprache, Anweisungen, Makro-Präprozessor, Beispiel
Assembler/Dokumentation/C8205_Standardmakros_c82ass.md	Übersicht der Standardmakros aus `c82ass.makros`: Aufruf, Wirkung, Zellenzahl
Assembler/Dokumentation/C8205_ZT_Makros_Uebersicht.md	Übersicht der ZT-Makros aus `macro_zt_loader.macro` (Projekt ZT-Loader)
Assembler/Claude/gen_lib/gen_lib_messung.md	Messung und Einsatzregel der Python-Bibliothek `c82gen.py` (Python als Makrosprache): ab wann sich ein Generator lohnt

C 8205 – Projektüberblick

Quelle: C8205-Interpreter-Projektueberblick.md · im Wortlaut der Quelle

Datei	Rolle
`Assembler/Regressionstest/LIESMICH.md`	Beschreibung der automatischen Prüfung des Assemblers (Regressionstest, `pruefe_alles.sh`, Negativtests, Fuzz); liegt nur im `Regressionstest.zip` des Projektstamms
`C8205Z/zt/Zusatztrommel_Zeitmodell.md`, `C8205Z/zt/ZT-Repository_Aufgabenstellung.md`, `C8205Z/zt/ZIL_Pruefung_Realisierbarkeit.md`	Prototyp C8205Z (Abschnitt 3b): Zeitmodell und Modulspezifikation der Zusatztrommeln, ZT-Repository und Urlader, Prüfung der Realisierbarkeit des ZIL-Ablaufs (historisch)
`C8205Z/pult_z/aufgabe_pult.md`	Arbeitsstand der Zeichnung des Bedienpults der Z-Ausführung (ruht)
`ton/SW_Ton_Beschreibung.md`	Geräusche von Schreibwerk und Lochstreifenleser für GRAFIK (nur im Gesamt-Zip)
`C8205-Interpreter-PDF.md`	Gestaltungsrichtlinie für alle PDF-Ausgaben; Dateibenennung und Liste der erzeugten PDFs (Abschnitt 8)

Alle Dateien werden zwischen Sitzungen erneut hochgeladen, da sie sich auch außerhalb einzelner Gespräche weiterentwickeln.

Jede Datei trägt eine Versionsnummer (`Version: y.xx-zzz`). Wer eine Datei ändert, erhöht deren Build (`zzz`) um 1; Hauptversion und Unterversion werden nur auf ausdrückliche Aufforderung des Projektinhabers erhöht (`C8205-Versionierung.md`).

Code-Dateien (kein Regelwerk, sondern dessen Umsetzung bzw. Werkzeuge dafür):

Datei	Rolle
`c8205_interp.py`	Der Python-Interpreter selbst (Abschnitt 2): liest Knotengraph und T2 zur Laufzeit aus den .md-Dateien, führt Tests aus
`testscripte/c8205_befehlssweep.py`	Vollständigkeitstest über alle 512 Operationscodes (Abschnitt 5)
`testscripte/erzeuge_leere_trommel.py`	Erzeugt bzw. setzt die Trommeldatei `Trommel/trommel.txt` auf eine leere Trommel zurück (alle 4096 Zellen 0)
`Hilfsprogramme/c8205_lochstreifen_anzeige.py`	Hilfsprogramm: schreibt einen virtuellen Lochstreifen als Klartext in einem Fenster aus (Abschnitt 4)
`Hilfsprogramme/c8205_lochstreifen_eingabe.py`	Hilfsprogramm: erzeugt einen virtuellen Lochstreifen aus Tastatureingaben (Abschnitt 4)
`Hilfsprogramme/c8205_lochstreifen_umwandlung.py`	Hilfsprogramm: wandelt eine Textdatei in einen virtuellen Lochstreifen um und umgekehrt, ohne Fenster (Abschnitt 4)
`Assembler/Sourcecode/` (C-Dateien)	C-Quellcode des Assemblers `c82ass` selbst (Abschnitt 3)
`Assembler/asm_und_test.sh`	Standardweg für ein Assemblerprogramm: assemblieren, `.test` ergänzen, Interpreter starten (Abschnitt 3)
`Assembler/Claude/gen_lib/c82gen.py`	Python-Bibliothek für Assembler-Generatoren (Abschnitt 3, Programmprojekte und Generatoren)
`c8205_grafik.py`, `grafik-interpreter-module/`, `c-module/`	Modus GRAFIK: Treiber, Python-Module (Pult, Schreibwerk, Geräte, Töne, Takt) und C-Module (Trommel, Bahnsperre, Unterbrechung, Tonmischer) (Abschnitt 4a)
`c8205_zt.py`, `C8205Z/zt_modul/`	Anbindung der Zusatztrommeln (Prototyp C8205Z) an den Interpreter und ihr C-Modul (Abschnitt 3b)
`lieferung_packen.sh`, `c8205_zip_packen.sh`	Packen einer Lieferung für KI-Sitzungen bzw. der Zips des Projektinhabers (`C8205-Projektstruktur.md`, Abschnitt 1)

7. PDF-Dokumente im Überblick

Zusätzlich zu den .md-Quellen liegen gut lesbare, A4-taugliche PDF-Ausgaben vor (Ordner `PDF/`), gestaltet nach einheitlicher Richtlinie (`C8205-Interpreter-PDF.md`). Die PDFs sind fertige Dokumente und werden nicht automatisch aus den .md-Dateien nachgeführt; sie können daher einen älteren Stand haben als ihre Quelle (der Stand steht auf dem Deckblatt bzw. in der Kopfzeile). Dateibenennung und Quelldateien: PDF-Richtlinie, Abschnitt 8. Die schlanke Claude-Fassung des Zips (Name endet auf „Claude“) enthält keine PDFs; die Tabelle gilt für das Gesamt-Zip.

PDF	Inhalt
`C8205-Projektueberblick_A4.pdf`	dieses Dokument
`C8205-Einstieg_fuer_Neueinsteiger_A4.pdf`	Was das Projekt enthält, in welcher Reihenfolge man liest und wo man bei einer Frage weiterliest
`C8205-Ueberblick_und_Gesamtaufbau_A4.pdf`	Einordnung, Geräteaufbau, Rechen- und Leitwerk, die vier Phasen der Befehlsabarbeitung (im Wortlaut der Quelle)
`C8205-Datenstruktur_und_Verarbeitungseinheiten_A4.pdf`	Register, Flags, Maschinenwort- und Befehlswortaufbau, Zahlendarstellungen
`C8205-Rechnerbeschreibung_A4.pdf`	Aufbau, Register, Maschinenwort, Magnettrommel, Befehlszyklus und Befehlsarten des Rechners (redaktionell)
`C8205-Bedienpult_und_Peripherie_A4.pdf`	Tasten, Schalter, Lampen, Kanäle, Lochstreifenleser, -stanzer und Schreibwerk; Token der Tastenfolge (redaktionell)
`C8205-Bedienungsanleitung_Interpreter_A4.pdf`	Bedienung von `c8205_interp.py`: Aufruf, Schalter, Geräte (redaktionell)
`C8205-Testhandbuch_A4.pdf`	Kurzanleitung „Test starten" (Claude- und Python-Interpreter), Tests schreiben und Protokolle lesen (redaktionell)
`C8205-Durchfuehrung_eines_Tests_A4.pdf`	Das Testformat im Wortlaut der Quelle: Trommeldatei, Tastenfolge, Testmarken, Prüfungen, virtuelle Geräte, Protokoll
`C8205-GRAFIK_Beschreibung_A4.pdf`	Der GRAFIK-Interpreter: Zweck, Fenster, Geräte, Töne, Aufruf und Grenzen im Wortlaut der Quelle
`C8205-GRAFIK_Pruefverfahren_A4.pdf`	Prüfverfahren des GRAFIK-Interpreters: Läufe, Tests, Abnahme im Wortlaut der Quelle
`C8205-Knotengraph_A4.pdf`	Der komplette Knotengraph als Diagramm, auf A4-Kacheln aufgeteilt
`C8205-Knotentabellen_A4.pdf`	Knotentabelle und Variablentabelle T2 im Wortlaut der Quellen
`C8205-Einzelbefehlsliste_A4.pdf`	Generische Befehlsbeschreibung und vollständige Einzelbefehlsübersicht
`C8205-Kodetabellen_A4.pdf`	Die Zeichenkodetabelle (Tabelle 1), Lochbild-Spalten als ●/○-Grafik
`C8205-Lochstreifen-Hilfsprogramme_A4.pdf`	Anzeige, Eingabe und Umwandlung virtueller Lochstreifen: Format, Aufruf, Zwischenablage und Installation
`C8205-Zusatztrommeln_A4.pdf`	Experimenteller Prototyp C8205Z: Eckdaten, Ablauf, Zeiten, Befehle und Modulspezifikation der Zusatztrommeln
`C8205-Testuebersicht_A4.pdf`	Alle Tests mit Inhalt, Geräte- und Vergleichsdateien und zugehörigem ASM-Quelltext; liegt in `Testausfuehrungen/`
`C8205-BIS_A4.pdf`	Bausteininterpretiersystem: Bausteine, KEP und Minimalprogramm; liegt in `Bausteininterpretiersystem BIS/markdown-dateien/`
`C8205-Standardmakros_A4.pdf`, `C8205-ZT_Makros_A4.pdf`	Standardmakros aus `c82ass.makros` und ZT-Makros aus `macro_zt_loader.macro`; liegen in `Assembler/Dokumentation/`
`C8205-Projektstruktur_A4.pdf`	Verzeichnisse, Skripte und erzeugte Dateien des Gesamtprojekts: welcher Pfad wofür zuständig ist
`C8205-PDF_Gestaltungsrichtlinie_A4.pdf`	Seitenformat, Folgeseiten, Deckblatt, Farben und Kachelung aller PDF-Ausgaben
`C8205-Assemblerdokumentation_A4.pdf`	Assembler `c82ass`: Aufruf, Sprache, Anweisungen, Makros, Beispiel; liegt in `Assembler/Dokumentation/` (im Wortlaut der `.md`)

8. Stand und offene Punkte

Dieser Überblick nennt keine Versionsnummern von Assembler, Interpreter oder Zip; maßgeblich sind der Dateiname des Zips, `c82ass --version`, die Zeile `Version:` der jeweiligen Datei und `C8205-Versionierung.md`. Den laufenden Arbeitsstand hält die Datei `aufgabe` im Zip-Stamm fest (eine leere Datei ist gewollt: dann steht der Arbeitsstand nur in der Nachricht des Inhabers). Die Gesamtversion eines Releases (Form `V.M.P`) steht im Namen des Zips und in `C8205-Versionierung.md`, Abschnitt 8. Es gibt zwei Zip-Fassungen: das Gesamt-Zip und die schlanke Claude-Fassung (ohne PDFs, `pult/`, `ton/` und einiges mehr; Programmierhandbuch, Abschnitt 11).

- **Prüfläufe:** Die Regressionstests laufen mit `testscripte/compute_all_tests.sh` (Interpreter), `Assembler/Regressionstest/` (Assembler), `testscripte/grafik_alle_tests.sh` (GRAFIK) und `C8205Z/Tests_Z/zt_tests.sh` (Zusatztrommeln).
- **BIS:** Dokumentiert sind die allgemeine Beschreibung, das KEP und das Minimalprogramm; weitere Bausteine sind noch nicht erfasst (Abschnitt 3a). Der Ordner des BIS besteht nur aus Dokumentation; das Minimalprogramm (v6, v7) liegt als Assemblerprojekt in `Assembler/Projekt_2/`.
- **Prototyp C8205Z:** experimentell (Abschnitt 3b); die Zeichnung des Bedienpults der Z-Ausführung ruht und wartet auf Fotos (`C8205Z/pult_z/`).
- **PDFs:** Sie werden nicht automatisch aus den `.md`-Dateien nachgeführt und können älter sein als ihre Quelle (Abschnitt 7).
- **Offene Punkte der Fachdokumente:** Sie stehen dort, nicht hier: Projektstruktur (Abschnitt 4: derzeit keine), Überblick und Gesamtaufbau (Abschnitt 8), GRAFIK-Beschreibung (Abschnitt 10) und Programmierhandbuch (Abschnitt 9). Die Versionierung hat keinen eigenen Abschnitt mit offenen Punkten.